

企业级软件工程 多 Agent 协同运作指导手册

三层架构体系——战略层 · 项目群层 · 战术层
22 个角色 · 26 个 skills · 端到端规范化交付

本手册沉淀了一套面向企业级软件工程智能化交付的三层多 Agent 协同体系：L3 战略层（企业架构、技术战略、投资治理、GRC 治理监督）、L2 项目群层（多项目协同、工程标准化、交付效能度量、数据治理）、L1 战术层（14 个专家子智能体覆盖需求→交付→数据闭环全生命周期）。涵盖设计方法论、三层协作机制、商用治理（可靠性/成本/可观测/集成）、落地实现步骤与一线实战经验，可作为企业级标准复用。

目录

- 一、为什么需要多 Agent 协同 03
- 二、三层架构体系总览 04
- 三、L3 战略层（3 角色 + 治理监督） 06
- 四、L2 项目群层（4 角色 + 横向能力） 08
- 五、L1 战术层（14 角色 + 编排者方法论） 11
- 六、流程沉淀与数据治理闭环 16
- 七、商用治理体系（五道坎 + 四元能力） 18
- 八、能力插件机制与防膨胀 21
- 九、从 0 到 1 落地：六步实施法 22
- 十、踩坑实录与反模式 24
- 十一、适用性、IDE 迁移与小结 26

一、为什么需要多 Agent 协同

单 Agent 模式下，一个 AI 同时扮演需求、设计、编码、测试、部署所有角色，企业级场景下系统性痛点：

痛点	表现	后果
角色串味	写完代码自己评审，等于没评审	质量门禁失效
上下文爆炸	一个会话塞 PRD+代码+测试，越往后越糊	后半程产出质量陡降
质量无门禁	没有强制卡点，AI 自报"已完成"就算完成	虚报、纸面通过、假修复被部署
经验不复用	每次都从零开始，踩过的坑反复踩	团队水平不沉淀
协作不可审计	改了什么、谁改的、为何改，全凭一张嘴	无法追溯、无法复盘
专业维度缺失	一个全栈 AI 兼顾不了安全/性能/数据/治理	企业级质量短板

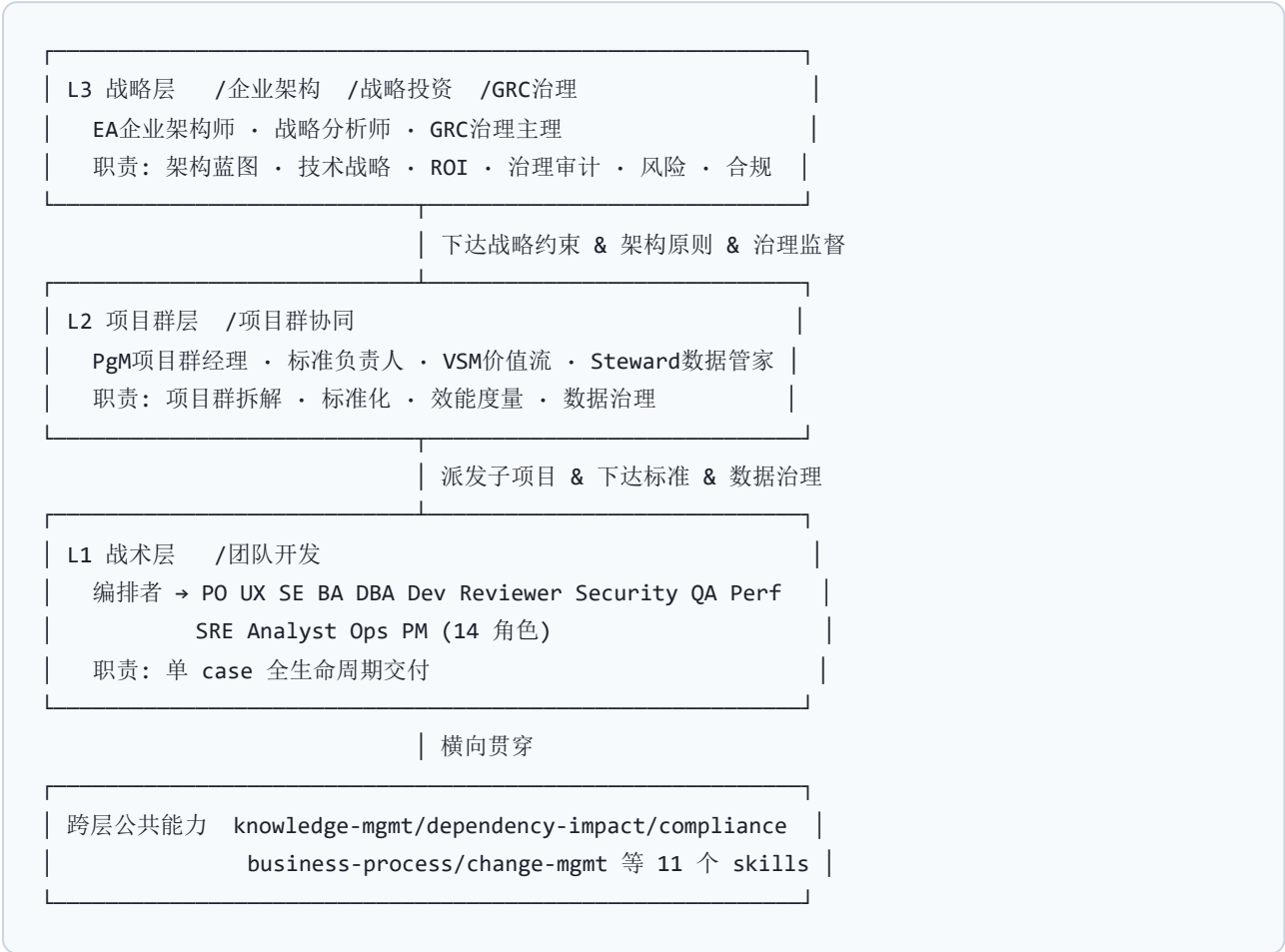
核心思想：把一个全能 AI 拆成有明确分工的专家子智能体，由编排者统一调度，按流水线交付。配合强制门禁、模型隔离、磁盘事实核对、商用治理，让质量不靠自觉——从“能用”升级到“企业敢用、能持续经营”。

本手册的定位

不是某个项目配置文档，而是一套**与具体项目、具体 IDE 解耦**的企业级协同范式。所有项目专属约定（技术栈、包结构、构建工具）不硬编码，由项目级 `CLAUDE.md` 运行时注入——同一套配置可用于任何规模项目。

二、三层架构体系总览

单层协同（只有 L1）只能完成"一个需求到一个交付"。企业级大型项目/变革项目需要**自上而下协同、端到端规范化**——这要求在 L1 之上叠加 L2（项目群层）和 L3（战略层）。



三层定位与指挥链

层级	回答的问题	"编排者"	调度对象	触发
L3 战略	做不做？投多少？架构怎么定？合规吗？	EA / Strategy / GRC	定义原则给 L2	/企业架构 /战略投资 定期GRC
L2 项目群	多项目怎么协同？标准？数据？	PgM	多个 L1 编排者	/项目群协同
L1 战术	一个需求怎么交付？	team-orchestrator	14 个角色	/团队开发

指挥链清晰解耦： EA/Strategy/GRC → PgM → 编排者 → 14 角色。每层只调度下一层，不跨层。三层独立命令触发，用哪层调哪层。

体系规模

22 个角色 + 26 个 skills（含 mlops 插件）+ 4 个斜杠命令。L3 战略层 3 角色，L2 项目群层 4 角色，L1 战术层 14 角色（核心 9 + 专项 5）+ 1 插件（mlops）。

三、L3 战略层（3 角色）

L3 是企业级变革项目的起点——不是"做什么需求"，而是"为什么做、做不做、架构怎么定、合不合规、风险可控吗"。向下传导架构原则与治理监督。

EA · 企业架构师

opus · /企业架构

定位：定义整个企业架构蓝图与演进路线，是**流程架构的主导者**（TOGAF 业务架构层）。

产出：业务/应用/数据/技术四层架构蓝图、架构原则集、演进路线、ADR、流程架构（调用 business-process skill）。

关键：与 Steward 协同（EA 定数据架构方向，Steward 落地治理）；流程 gap 衍生变革项目给 PgM。

Strategy · 战略投资分析师

opus · /战略投资

定位：回答"做不做、投多少、先做哪个、值不值"。看商业价值与技术投资，非产品行为数据。

产出：可行性研究（技术/经济/运营/时间四维）、ROI 商业案例、投资优先级、技术雷达（Adopt/Trial/Assess/Hold）。

人机边界：自主产出方案；立项与预算分配保留人工确认。

GRC · 治理/风险/合规主理

opus · 治理监督层

定位：体系"治理闭环的兜底人"。基于 COBIT 2019/COSO ERM/ITIL 4/ISO 31000。是体系的"审计委员会+风控委员会"。

产出：治理审视报告、治理审计报告、企业风险登记册、控制有效性评估、合规状态报告。

六职能：治理策略审视 / 治理审计 / 企业风险评估 / 合规治理 / 控制有效性评估 / 持续改进驱动。

人机边界（硬约束）：AI 做数据采集/规则检查/风险预警/报告；**审计签字/内控认证/处罚决定/重大风险处置保留人工**（法定人责）。

边界：不下场执行（EA/Standards/Reviewer 执行），不替代法定人责。

L3 skills

enterprise-architecture（四层架构/ADR） · tech-radar（技术雷达/ROI） · enterprise-grc（COBIT/COSO/ITIL 裁剪）

四、L2 项目群层（4 角色）

L2 是当前体系**最大的协同枢纽**——单编排者只管一条线，企业级是多项目并行、相互依赖、共享资源、需统一标准与数据治理。L2 的 PgM 调度多个 L1 编排者。

PgM • 项目群经理

opus · L2 的"编排者" · /项目群协同

定位：像 L1 编排者调度 14 角色，PgM 调度**多个 team-orchestrator**。把战略拆成项目群，派发子项目，协调依赖与里程碑。

产出：项目群路线图、依赖图、里程碑计划（milestones.md）、资源分配、跨项目冲突裁决。

Standards • 方法论标准负责人

sonnet · 唯一"定义规范"角色

定位：整套体系唯一"定义规范"的角色——其他都"遵循规范"，本角色"编写维护规范"。

产出：CLAUDE.md 治理、工程标准手册、质量门禁定义、模板库、API 契约规范。把 EA 架构原则翻译成 L1 可执行标准。

VSM • 价值流分析师

sonnet · PgM 的"眼睛"

定位：DORA 四指标 + 流动效率度量整个项目群交付效能，识别瓶颈。是 PgM 的"眼睛"。

产出：DORA 报告（部署频率/变更前置时间/变更失败率/恢复时间）、流动效率、瓶颈诊断。

Steward • 数据管家

sonnet · 企业数据资产主理

定位：企业级数据资产**主理人**——管数据的治理与价值化。数据治理失败头号原因是无主理人，本角色是兜底责任人。

产出：数据标准、质量规则、元数据字典、主数据管理方案、数据目录、数据资产化方案。

边界：与 DBA（数据存储）/Analyst（产品度量）/EA（架构方向）零重叠。Steward 管治理与价值化。

L2 skills

program-management（项目群治理+里程碑协议） · engineering-standards（标准翻译） · devops-metrics（DORA） · data-governance（DAMA 裁剪）

五、L1 战术层（14 角色）

5.1 核心角色（9 个，全程参与）

PO · 产品经理

opus

职责：PRD+验收标准（必须可测）。**强化：**产品管理（路线图/RICE 优先级/竞品，调用 product-management）+ 需求工程（全生命周期/变更控制/追溯，调用 requirements-engineering）。不只是 PRD 输出员，是产品管理者。

UX · 交互设计师

opus

职责：PRD 转 ASCII 线框图 + 结构化界面规格，开发前与用户对齐。

SE · 系统工程师（技术主导）

opus · 接编排者架构职责

定位：L1 战术层**技术主导**。编排者管流程，SE 管技术（架构职责从编排者剥离给 SE）。

职责：技术方案设计（阶段1.8）、开发全程质量主导、关键技术问题攻坚。

边界：不写业务代码（Dev）、不做规范检查（Reviewer）。SE 纵向贯穿技术全程，其他专项角色横向切片。

BA · 需求分析员

sonnet

职责：PRD 拆技术任务清单（标类型/依赖/接口契约/可独立验收）+ 变更影响评估。读 DBA 数据设计 + SE 技术方案。

Dev · 全栈开发

sonnet · 不提交 git

职责：按任务清单和接口契约、遵循项目编码规范写代码。**防虚报：**回报告前 git diff --stat 自证改动落地。

5.1 核心角色（续）

Reviewer • 代码评审

opus · 须与 Dev 不同模型

职责：磁盘事实核对（git diff 与 Dev 报告比对）+ 规范符合度门禁。与 Dev 模型不同、角色独立、互不通信。

QA • 测试工程师

sonnet

职责：按 AC 反推用例（正常/边界/异常）。**磁盘事实核对：**预期基于磁盘真实代码推演，不抄 Dev 报告。

Ops • 项目助手（平台工程）

haiku · 唯一 git commit 角色

职责：部署/迁移/构建执行、git 归档（唯一执行 commit）、配置同步。**强化：**平台工程（IDP/黄金路径/CI-CD，调用 platform-engineering）+ ITSM（事件/问题/变更/SLA，调用 it-service-management）。

PM • 项目经理

sonnet · 纯项目管理

职责：变更日志（changelog，最高优先——任何需求先登记）、看板、风险日志、进度播报、闭环跟踪。**纯项目管理**，不含架构职责（已剥离给 SE）。

5.2 专项角色（5 个，按需介入）

DBA • 数据库工程师

sonnet · ★ 涉数据层时

介入：阶段1.6（BA 前置输入）。**职责：**多存储选型、关系型建模、索引、读写分离/分库分表、迁移幂等。

Security • 安全工程师

opus · ★ 须与 Dev 不同模型

介入：阶段3.5 与 Reviewer 并行。**职责：**鉴权越权/注入/敏感信息/依赖漏洞/威胁建模（STRIDE）。

Performance • 性能工程师

sonnet · ★ 高并发路径时

介入：阶段4 与 QA 接力。**职责：**压测方案、性能基线、N+1/无索引/连接池/锁诊断、调优建议。

5.2 专项角色 (续)

SRE • 可靠性工程师

sonnet • ★ 上线前/高可用时

介入: 阶段4.5。**职责:** 可观测性三支柱 (Metrics/Logging/Tracing)、SLI/SLO、容量、熔断限流降级、混沌、事故复盘。

Analyst • 产品分析师

sonnet • ★ 双阶段

介入: 开发期埋点 (阶段1.7) + 交付后数据分析反哺 PO (阶段6)。**强化:** 与 Steward 协同做企业级数据价值挖掘。

职责: 指标体系 (北极星/AARRR/HEART/OSM)、埋点、漏斗/留存/路径、A/B 实验、迭代建议。

MLE • 机器学习工程师 (插件)

sonnet • mlops 插件

定位: 模型生命周期工程化。涉 AI 产品时激活 (按需 `install --plugin mlops`)。

职责: 模型训练/微调工程化、模型评测 (Eval)、部署/灰度/回滚、监控 (漂移/幻觉/成本)、Prompt/RAG 版本化。

5.3 协作流程 (standard 档完整流程)

[需求/bug 输入]

- 阶段-1 变更登记 (派 PM)
- 阶段0 理解 (编排者亲自做: 复述+澄清+定档位+建case)
- 阶段1 PRD (派 PO)
- 阶段1.5 交互设计 (派 UX) ← 涉界面时强制
- 阶段1.6 数据建模 (派 DBA) ★ ← 涉数据层时
- 阶段1.7 度量埋点 (派 Analyst) ★ ← 大型产品时
- 阶段1.8 技术方案 (派 SE) ← standard 档强制
- 阶段2 任务拆解 (派 BA, 读 DBA + SE 产出)
- 阶段3 开发 (派 Dev)
- 阶段3.5 多重评审 (并行, 全过才放行)
 - └ 代码评审 (Reviewer 查规范)
 - └ 安全评审 (Security 查安全) ★
- 阶段4 测试 (QA + Performance 接力)
 - └ 功能正确性 (QA)
 - └ 性能基线 (Performance) ★
- 阶段4.5 可靠性设计 (派 SRE) ★ ← 上线前强制
- 阶段5 文档/部署 (派 Ops) └ git 归档(全门禁过)
- 交付验收
- 阶段6 数据闭环 (派 Analyst) ★ ← 反哺 PO 下一轮 PRD

fast 档：小改动（文件≤5/AC≤3/不涉DDL/不跨模块）走轻流程，跳过 UX/独立BA/部分文档。

六、流程沉淀与数据治理闭环

企业的核心是**通过运作、经验和业务锻造沉淀体系和能力**。三大沉淀要素：流程、IT 产品、数据。本体系对这三块的承接：

沉淀要素	承接角色/skill	价值
流程	EA 主导 (business-process skill) + PgM 衍生项目	标准化流程指导企业作战
IT 产品	L1 全流程交付 + Ops 平台工程	流程的 IT 落地载体
数据	Steward 主理 (data-governance skill) + Analyst 挖掘	企业长期价值壁垒

6.1 流程沉淀：战略→架构→流程→项目衍生

关键认知纠正：流程设计不是"架构之后"单独一步，而是**架构之内**——它是企业架构的业务架构层 (TOGAF)。流程设计天然归 EA。

EA 调用 **business-process skill** 设计流程架构 (L1 价值流→L2 流程→L3 子流程→L4 活动，对标 APQC, BPMN 2.0 建模)，沉淀到流程资产库。然后做**流程→IT 映射 gap 分析**——哪些流程缺 IT 支撑/需优化 → 这些 gap 成为 PgM 拆变革项目的输入。**变革项目不需要单独机制**，从流程 gap 自然衍生，PgM 承接。

6.2 变革管理：补"人的层面"

变革项目 70% 失败率，主因"只管系统不管人"。**change-management skill** 补变革的"人的层面"——Prosci ADKAR + Kotter 八步法 + 利益相关者分析 + 沟通/培训/阻力化解。EA/PgM 调用，**不新增角色**。

6.3 数据治理：企业长期价值壁垒 (Steward 主理)

数据是企业最终转化物，能体现多少价值代表体系成败。当前体系原本 DBA/Analyst/EA 都覆盖不了企业级数据治理——**这是真空白**。数据治理失败的头号原因是"无主理人"，分散必落空。

DAMA 治理域	Steward 职责
数据标准化	主数据标准、数据元标准、口径字典 (跨系统统一)
数据质量	完整性/准确性/一致性/及时性，规则与监控
元数据管理	数据字典、数据血缘、影响分析
主数据管理	核心实体单一真相源，跨系统分发同步
数据资产化	数据目录、数据产品、数据服务化

数据价值挖掘	与 Analyst 协同，跨产品业务洞察
数据合规	隐私/留存/GDPR，与 compliance-audit 协同

价值闭环

治理（标准化/质量）→ 资产化（可检索可复用）→ 价值挖掘（数据驱动业务）→ 反哺治理（发现质量问题回流）。数据从"负债"变"资产"再变"竞争力"。

七、商用治理体系（五道坎 + 四元能力）

核心认知：商用卡住的地方几乎都不在功能层。可靠性、安全、成本、可观测、集成——这五道坎和四个元能力，是体系从“概念车”变“可量产的车”的关键。全部已落地为体系治理类 skill（meta 层，默认启用）。

7.1 五道商用坎（对应 skill）

坎 1：AI 可靠性治理 → reliability-governance（商用门槛）

从点状防虚报升级为**体系级四道防线**：

防线	内容
① 不可逆操作人工锁	生产部署/DDL/数据删除/配置变更/主分支push/下线/模型上线——7 类操作一律用户确认才执行
② 输出兜底校验	关键产出独立校验（产出者不自证、校验者不抄作业、编排者兜底抽查）
③ 全链路可回滚	部署前必产出回滚方案，无方案不部署；迁移必配套回滚
④ 置信度评估	低置信打回或上报，不自作主张放行；宁可慢不可假

坎 2：安全合规 → compliance-audit + Security

等保/GDPR/行业合规检查与审计追溯链。Security 查产品代码安全，compliance-audit 查合规审计链。

坎 3：成本治理 → cost-governance（商用可持续）

手段	内容
token 预算与计费	每 case 设上限，80% 预警，100% 停止派生。各角色消耗可计量
模型动态降级	简单任务降 mechanical， reasoning 档不降 （降了门禁失效）
上下文裁剪	按角色职责裁剪到“本次必需”最小集，不无脑全量注入

7.1 五道商用坎（续）

坎 4：体系自身可观测性 → system-observability（长期运营）

SRE 监控产品系统，本 skill 监控**体系自身健康**：

维度	指标	异常信号
case 成功率	一次通过率	走低说明流程有问题
角色产出质量	各角色打回率	突增→定义可能有问题
门禁有效性	Reviewer/Security 漏判率	漏判→门禁清单要补
经验库健康	去重率/复用率	污染→清洗
token 消耗	各角色分布	飙升→上下文注入有问题

数据反哺方法论标准负责人做**体系自身迭代**——像产品发版一样优化体系。

坎 5：落地集成 → data-contract（适配企业工具链）

企业已有 Jira/Confluence/GitLab，不可能换工具。解法：**数据契约标准化**——产出用业界标准格式，换工具是导出导入，不是重写。

数据类	契约格式	迁移代价
文档型（PRD/设计/标准）	Markdown + YAML frontmatter	低（任何 Wiki 导入）
项目群管理（里程碑/任务）	JSON Schema + REST API	中（Jira/Plane 支持）
接口契约	OpenAPI 3.0	低（Stoplight/Swagger 全支持）
度量数据	Prometheus + OpenTelemetry	中（Grafana/Datadog 吃这套）

开源起步 → 商业升级路径

阶段 0：全 git markdown（当前，产出已符合契约）→ 阶段 1：Gitea+Outline+Plane+Stoplight+Grafana（开源起步）→ 阶段2/3：按需换 Jira/Confluence（数据导出导入，契约不变）。

7.2 四个元能力（适配未来，保持领先）

领先性不是"现在功能最多"，是"进化最快"。没人能预测 2027 的范式，但可建立让体系快速吸收新范式的能力。

元能力	对应 skill	怎么保持领先
① 模型无关性	model-registry	角色写能力档位（reasoning/structured/mechanical），路由表映射模型。换代只改路由表，不改 22 角色
② 框架无关性	方法论层（已具备）	编排者方法论是 markdown prompt，不绑 Agent 框架。换 LangGraph/CrewAI 不影响方法论
③ 协议标准化	mcp-integration	用 MCP 标准对接工具，未来任何支持 MCP 的工具即插即用
④ 进化机制	capability-registry + 三问闸门	新能力能有序接入而不膨胀；未通过三问拒绝注册

7.3 体系治理 skill 全景（meta 层，11 个）

这 11 个不是业务能力，是**体系自身可靠运转的治理机制**，默认启用：

skill	作用	解决
capability-registry	能力注册表 + 三问闸门	防膨胀
model-registry	模型路由表（档位→模型）	防过时
reliability-governance	四道防线	商用门槛
cost-governance	token 预算 + 降级 + 裁剪	商用可持续
data-contract	标准化契约	落地集成
system-observability	体系健康监控	长期运营
mcp-integration	MCP 标准对接	适配未来
business-process	BPM 流程架构	流程沉淀
change-management	OCM 变革管理	人的层面
knowledge-management	组织级知识库	知识资产
dependency-impact	全局依赖影响	跨项目协调

诚实边界：治理 skill 定义的是**规则与协议**（方法论层）。让 AI 出错从"不可控"变"可控可回滚可审计"，但不等于零风险——AI 可靠性是概率性的，最终生产把关责任在人。运行时真实计量/采集/代码对接依赖框架照规范实现。

八、能力插件机制与防膨胀

角色和 skills 无限制膨胀是体系最大的慢性病。本章的**插件机制**是根本解法：让体系可自演进，但不膨胀。

8.1 三类能力分级（防膨胀第一道闸）

类型	含义	安装	可禁用
core	软件工程刚需的核心能力	默认装	✗
layer	三层架构的 L2/L3 层级能力	默认装	✓ 纯 L1 可禁
plugin	可选领域插件（MLOps 等）	按需装	✓ 默认就不装

8.2 三问闸门（防膨胀硬约束）

三问闸门

1. 解决什么缺口？——现有角色/skill 覆盖不了什么？若现有能覆盖 → 拒绝注册。
2. 与现有角色的边界在哪？——职责如何划分，不串味？边界不清 → 拒绝注册。
3. 什么条件下才激活？——trigger_when 必须明确，不是每次都派。无明确触发 → 拒绝注册。

8.3 当前已注册插件

插件	角色	trigger_when	安装
mlops	team-mle	涉模型训练/微调、RAG、Prompt 工程规模化、LLM Agent 平台	<code>install --plugin mlops</code>

其他领域（FinOps/平台工程/数据契约/需求工程等）已作为**横向 skill**纳入默认能力（不占 plugin 名额）。新领域按三问流程注册。

8.4 注册新插件流程

1. 编排者遇注册表未覆盖的领域 → **不擅自造角色**
2. 上报方法论标准负责人走三问评估
3. 三问通过 → 注册 capability-registry + 建角色/skill + install PluginMap 登记
4. 三问不过 → 复用现有能力
5. 用户按需 `--plugin <名>` 安装

这条机制让体系从“每次追新都要大改”变成“注册一个插件即可接入”——可演进、可扩展、但不膨胀。

九、从 0 到 1 落地：六步实施法

第 1 步：规划角色与模型分工

先定花名册。起步可只配核心 9 角色（PO/UX/SE/BA/Dev/Reviewer/QA/Ops/PM），跑通后加专项（DBA/Security/Performance/SRE/Analyst），企业级再加 L2/L3。

档位	典型角色	模型
reasoning	PO/UX/SE/Reviewer/Security/EA/PgM/Strategy/GRC	opus（高端）
structured	BA/DBA/QA/Performance/SRE/Analyst/Steward/Standards/VSM/MLE	sonnet（中端）
mechanical	Ops	haiku（低端）

模型隔离硬约束：Reviewer 和 Security 必须与 Dev 不同模型。

第 2 步：创建工程内配置目录

```
你的工程/
├── agents-config/
│   ├── README.md / install.sh / install.ps1
│   ├── agents/          22 个 team-*.md
│   ├── commands/        斜杠命令（/团队开发 /项目群协同 /企业架构 /战略投资）
│   └── skills/           26 个 skills（含编排者方法论）
```

第 3 步：编写角色子智能体定义

每个角色 .md = front matter（YAML）+ 角色 Prompt。每个 Prompt 必须写清：“不做”边界（防串味）、“只做”职责、磁盘事实核对段（评审/测试类）、项目约定占位（CLAUDE.md 注入）、本次经验沉淀段。

第 4 步：编写编排者方法论（核心）

含：触发约定、商用治理（路由表+可靠性四道防线）、核心职责、协作流程、专项角色触发规则、多重门禁、自治上报、模型隔离、产物落盘、经验记忆闭环。

第 5 步：编写一键安装脚本

默认装 core+layer，plugin 按需 `--plugin`。Windows 的 install.ps1 须 **UTF-8 with BOM**（防中文 PS5 崩溃）。

第 6 步：验证与首次使用

1. 跑安装脚本 → 重启 IDE → 设置→子智能体看 22 个 team-* → /团队开发 测试一下 验证编排者

十、踩坑实录与反模式

坑	现象	解法
编排者越权	自己直接跑部署，不派 Ops	方法论硬写"不越权执行"清单
自审自	Dev 和 Reviewer 同模型，评审形同虚设	强制模型隔离，派生时校验；Security 同理
经验不注入	子智能体反复踩相同坑	hook 不作用于子智能体，编排者必须手动注入
子智能体互相调用	调用栈爆炸、上下文污染	子智能体只由主 Agent 调度，禁止横向
技能不被调用	子智能体"不知道"有技能	技能由主 Agent 加载并注入 prompt
写提交不分	Dev 写完自己 commit，评审失效	git commit 唯一由 Ops 执行
编排者兼架构	编排者既调度又做技术方案，两头不深	架构职责剥离给 SE，编排者纯流程

10.2 虚报类故障（最高危）

Dev 虚报改动，Reviewer/QA 基于报告做纸面通过，假修复被部署——最危险故障。

根因：评审/测试角色采信了 Dev 报告文字，没独立核对磁盘事实。报告可幻觉，git diff 不会。

解法（贯穿四层）：Dev 回报告前 git diff 自证 → Reviewer/QA 第一步磁盘事实核对 → 编排者派 Ops 部署前抽查 → 任一层发现虚报立即停止、记入风险日志。

10.3 配置类故障

子智能体消失 / 技能重复 / install.ps1 崩溃 / model 冲突

子智能体消失/重复：IDE 同时加载工程级和用户级同名定义 → 删工程级 .zcode/agents|skills，只放 runtime/。

install.ps1 崩溃：UTF-8 无 BOM 在中文 PS5 按 GBK 解析崩溃 → 保存为 UTF-8 with BOM。

model 冲突：用户界面手选后回写 custom:uuid:name 格式是正常的，不要手改回内置别名。

10.4 排查"看不到/重复"标准流程

1. 查用户级 `~/ .zcode/` 数量齐全
2. 查工程级 `.zcode/` 有无 agents/skills（有就删，只留 runtime）
3. 查 model ID 有效

十一、适用性、IDE 迁移与小结

11.1 按项目类型裁剪

项目类型	推荐配置
内部工具/脚本	核心 5 (PO/Dev/Reviewer/QA/Ops)
标准业务系统	核心 9 (+UX/SE/BA/PM)
多存储/数据密集	核心 + DBA + Steward
高并发系统	核心 + DBA + Performance + SRE
对外服务/含支付	核心 + Security
大型 C 端产品	核心 + 全部专项 + Analyst
企业级变革项目	三层全开 (L1+L2+L3+GRC)
AI 产品	+ mlops 插件 (team-mle)

11.2 IDE 迁移

方法论与 IDE 解耦，迁移只替换：① 配置目录路径 ② 安装脚本目标路径 ③ 派生 API 调用形式。**编排者** 方法论、角色 Prompt、阶段流程、检查点规则、经验记忆机制、商用治理——这些是 IDE 无关的，可直接复用。

11.3 价值总结

维度	价值
分工	每个 AI 只干擅长的事，Prompt 短而精
门禁	强制检查点 + 模型隔离 + 磁盘核对，质量不靠自觉
可审计	产物落盘、case 目录、经验沉淀，全程留痕
可复用	配置随 Git 走，新人 clone 一键就绪
可迁移	方法论与 IDE 解耦，换工具不换思路
专业纵深	专项角色覆盖数据/安全/性能/可靠性/度量/治理
企业闭环	战略→架构→流程→数据→治理，端到端持续经营
防膨胀	插件机制 + 三问闸门，可演进但不失控

一句话：这套体系让企业级软件工程从"人盯人、开会协调"变成"数据驱动、机制保证"——架构原则自动传导，进度自动上报，依赖主动协调，冲突有明确路径，治理有闭环兜底。从战略到战术、从建设到治理、从当下到未来，每个层级各司其职，靠共享文档树和里程碑协议拉通。

本方案基于真实项目实践沉淀 + 业界标准 (TOGAF/COBIT/COSO/ITIL/DAMA/APQC/BPMN/ADKAR/DORA)，配置源码与脚本均可直接复用。